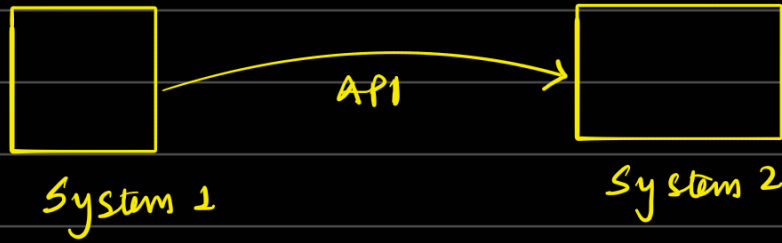


GRPC → Part - 1

1) API → Application Programme interface { two software }
Communicate



Example : REST, GraphQL, GRPC, SOAP etc.

2) REST → [Representational state transfer]

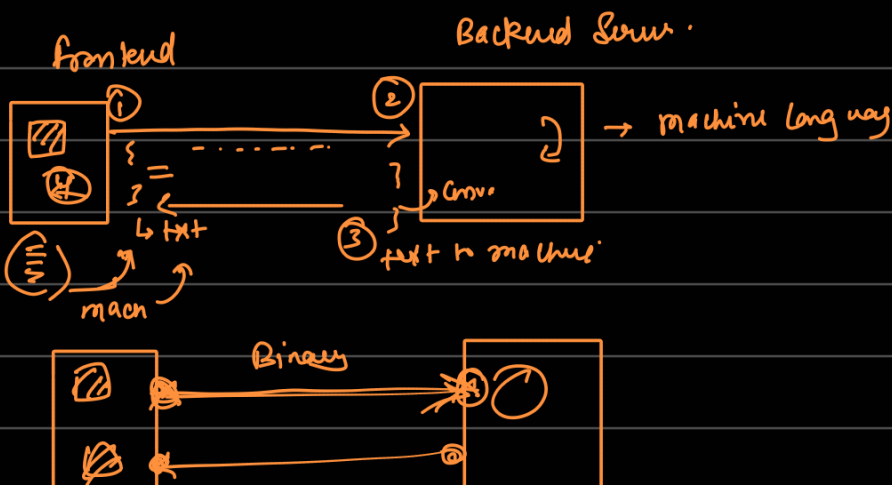
SOAP → XML → transfer data in format of
REST → JSON / XML → usually

Request : → Protocol → HTTP/1.1 (always)

{ email: " " } → Human Readable, slow { name: "Kashik" }
→ Streaming limited Age: "10"
2 + 10 + 20 = 22 bytes 2 + 4 + 3 + 5 + 17 ⇒ 21 bytes } Phone: "7830-..."

GRPC → [Google Remote Procedure call]

→ Bytes → Binary format - Protobuf
→ HTTP/2 → Support multiplexing / Streaming
→ Faster, Require more setup to read



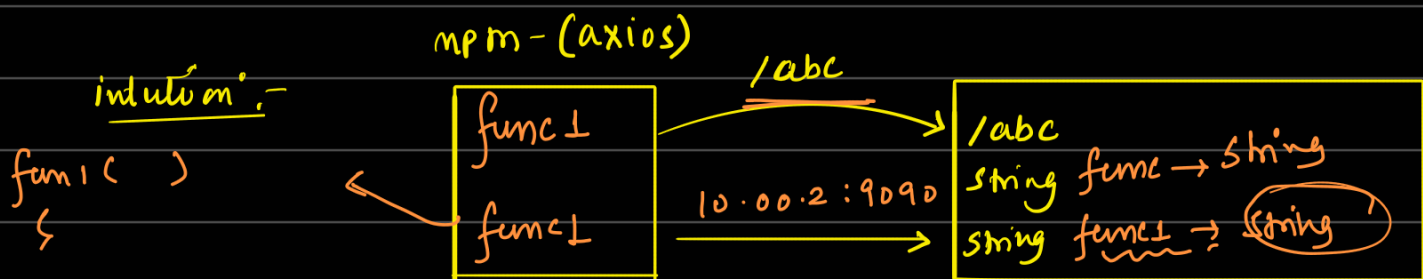
Why gRPC?

Rest	gRPC	Websocket	kafka
HTTP/1.1	HTTP/2	TCP	TCP
JSON/XML	Protobuf (Binary)	Text/Binary	Binary / JSON
No Streaming	Streaming	Streaming	different - arch.
Stateless	Stateless	Stateful	Persistent
Simple API, CRUD operation	microservice, internal API, cross language communication	live chat, online games, Google Docs	Event Driven System, real time analytics



gRPC - Part-2

introduction:-

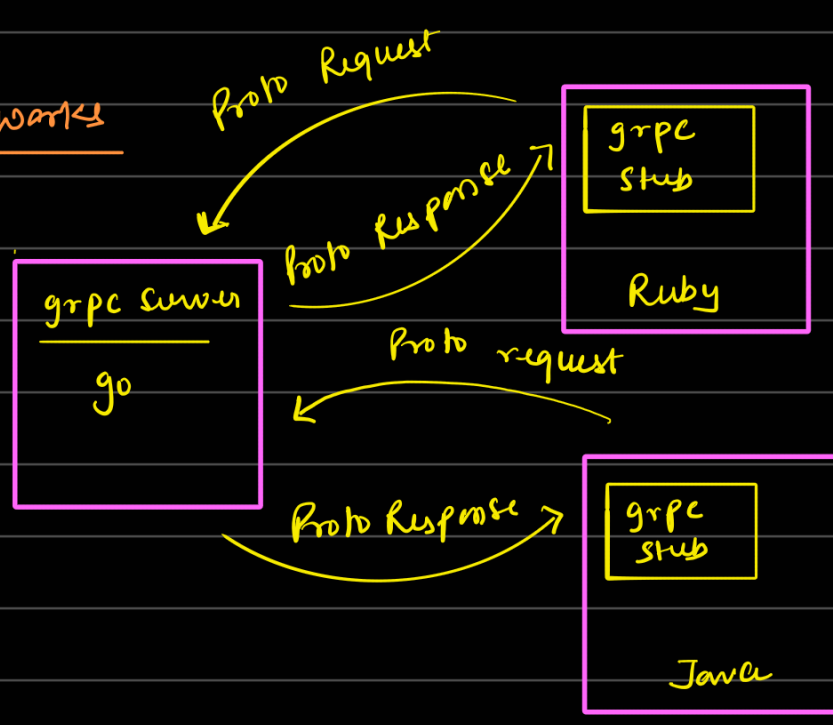


channel.func()
~~was~~
 = Start
 }

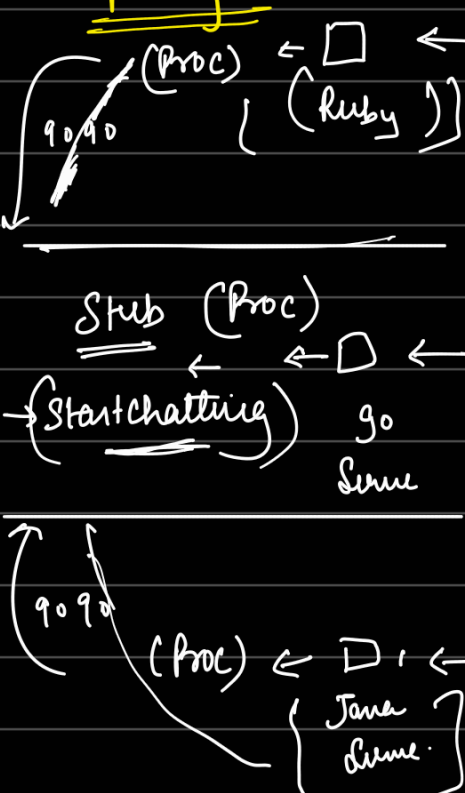
React
 website

Server Java Spring
 Boot

how it works



Proto file



```

syntax = "proto3"

message helloRequest {
    int id = 1;
}

message helloResponse {
    int id = 2;
}

service Chat {
    rpc StartChatting (helloRequest)
        returns (helloResponse);
}
  
```